

Introduction

In this assignment, I worked on building and running a simple full-stack application using Docker. The main objective was to understand how Docker helps in containerizing applications and running multiple services together using Docker Compose.

The project consists of:

- A **frontend** (running on port 3000)
 - A **backend (Flask API)** handling user input
 - Docker used to containerize and run both services
-

What I Implemented

First, I created a simple web application where the user enters their name. This input is sent to a Flask backend, which processes the request and returns a response.

Then, I containerized the application using Docker by:

- Writing a **Dockerfile** for the backend
 - Setting up **Docker Compose** to run services together
 - Exposing ports for frontend and backend communication
-

Running the Application

After setting up everything, I used the following command:

```
docker compose up --build
```

This command:

- Builds Docker images
- Starts containers

- Connects frontend and backend

Once the containers started successfully, I accessed the app in the browser.

Application Output

◆ Initial Page (Frontend UI)



localhost:3000

Enter Your Name

The application first displays a simple input form where the user can enter their name. This confirms that the frontend is working properly.

◆ After Submitting Input (Backend Response)



localhost:3000/submit

Enter Your Name

Hello Sai Chowdary, response from Flask backend!

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def helloworld():
    return "Hello World!"

@app.route("/get_data")
def getdata():
    return "Your data"

if __name__ == "__main__":
    app.run()
```

After entering the name and clicking submit, the request is sent to the Flask backend. The backend processes the input and returns a personalized message:

“Hello Sai Chowdary, response from Flask backend!”

This confirms:

- Backend is running inside Docker
- Communication between frontend and backend is successful

What I Learned

Through this assignment, I learned:

- How Docker containers work
- How to run multiple services using Docker Compose
- How frontend and backend communicate inside containers

- Importance of `.gitignore` (to avoid pushing `.venv`)
 - Handling Git issues like branches, push errors, and repo structure
-

Challenges Faced

While working on this assignment, I faced some issues:

- Git repository misconfiguration
- Folder structure issues (Docker Assignment not visible)
- Pushing code errors (`origin not found`, `fetch first`)
- Accidentally pushing `.venv` files

I resolved them by:

- Fixing Git remote
 - Removing `.venv`
 - Using `.gitignore` properly
 - Re-structuring the repository
-

Conclusion

This assignment helped me gain practical experience with Docker and Git. I now understand how to containerize applications and manage projects in GitHub properly. It also improved my debugging skills while handling real-world issues.